

Технічне завдання: Система обліку 3D-друку "Filamentary"

Дата: 25.05.2026

Версія: 1.0

Статус: Production-Ready

1. Загальний опис

Filamentary — це автономний програмний комплекс класу Local-First для обліку матеріалів та моніторингу 3D-принтерів.

- **Платформи:** Windows, Android (з можливістю розширення на Linux, macOS, iOS).
- **Архітектура:** Local-First. Дані зберігаються в локальній базі даних (SQLite), синхронізація відбувається через Google Drive шляхом обміну Журналом транзакцій (Audit Log).
- **Стек:** Flutter + SQLite (Drift) + Firebase Auth (Google Sign-In) + SQLCipher.
- **Позиціонування:** Повний контроль даних на пристрої з миттєвою синхронізацією через ваш особистий Google Drive без власного backend-сервера.

1.1 Візуалізація потоку даних

Потік транзакції в системі виконується за строгою черговістю для запобігання втрати даних в офлайн-режимі:

1. **Зміна в UI** → Користувач здійснює дію (наприклад, редагує принтер або додає катушку пластику).
2. **SQLite (запис + транзакція)** → Зміни негайно фіксуються в локальній базі даних за допомогою Drift.
3. **Queue (черга)** → Подія додається в локальну таблицю черги синхронізації (`sync_queue`) зі статусом `pending` .
4. **Sync Service (обмін логом з хмарою)** → Фоновий сервіс виходить на зв'язок з Google Drive API та надсилає оновлений шматок журналу.
5. **Merge (остаточне оновлення локальних баз)** → Обробка відповідей, злиття логів з іншими пристроями та фінальна актуалізація стану.

2. Ключові сутності (Моделі даних)

Кожна сутність містить обов'язкові службові поля `version` та `timestamp` для забезпечення коректної децентралізованої синхронізації та вирішення конфліктів.

1. **Printer:**

- `id` (UUID, String) — унікальний ідентифікатор пристрою.
- `name` (String) — користувацька назва принтера.
- `ipAddress` (String) — мережева адреса для підключення.
- `port` (Int) — порт підключення (за замовчуванням 80 або 8888).
- `apiKey` (String) — токен доступу до API.
- `model` (String) — модель принтера (наприклад, Bambu Lab P1S, Voron 2.4).
- `type` (Enum: Klipper, Bambu) — архітектурний тип інтеграції.
- `slotsCount` (Int) — кількість активних слотів подачі філаменту (1, 2, 4, 8).
- `activeSlotsJson` (String / JSON) — поточний мапінг заправлених катушок у слоти.
- `isDeleted` (Boolean) — прапор м'якого видалення.
- `version` (Int) — лічильник версії об'єкта.
- `timestamp` (DateTime) — точний час останньої модифікації.

2. **MaterialModel (Катушка філаменту):**

- `id` (UUID, String) — унікальний ідентифікатор катушки.
- `manufacturer` (String) — бренд (наприклад, Sunlu, PolyMaker).
- `type` (Enum: PLA, PETG, TPU, ABS, ASA) — тип полімеру.
- `color` (String) — назва кольору або HEX-код.
- `initialWeight` (Double) — початкова вага нетто пластику (грам).
- `usedWeight` (Double) — кількість витраченого plastic в грамах.
- `status` (Enum: active, finished) — поточний стан використання.
- `isDeleted` (Boolean) — прапор м'якого видалення.
- `version` (Int) — лічильник версії об'єкта.
- `timestamp` (DateTime) — точний час останньої модифікації.

3. **PrintJob (Історія друку):**

- `id` (UUID, String) — унікальний ідентифікатор завдання.
- `printerId` (FK, String) — прив'язка до принтера, який виконував друк.
- `modelName` (String) — назва роздрукованого G-code файлу.
- `startTime` (DateTime) — час запуску завдання.
- `duration` (Int) — тривалість друку в секундах.
- `usedMaterials` (String / Map<ID, Weight>) — JSON-структура витраченого пластику з конкретних катушок.

- `status` (Enum: `InProgress`, `Completed`, `Failed`) — фінальний статус друку.
- `isDeleted` (Boolean) — прапор м'якого видалення.
- `version` (Int) — лічильник версії об'єкта.
- `timestamp` (DateTime) — точний час останньої модифікації.

4. Transaction (Журнал дій):

- `id` (UUID, String) — ідентифікатор запису транзакції.
- `entityId` (String) — зовнішній ключ до сутності (Printer або MaterialModel).
- `changeValue` (Double) — дельта зміни (наприклад, ручна корекція ваги `-50g`).
- `type` (Enum: `WriteOff`, `Refill`, `Correction`) — тип бухгалтерської операції.
- `timestamp` (DateTime) — час фіксації транзакції.

2.1 Політика видалення та цілісності даних

- **Soft Delete (М'яке видалення):** Для всіх сутностей (Printer, MaterialModel, PrintJob) використовується прапор `isDeleted` . Видалення об'єкта користувачем в інтерфейсі не є фізичним очищенням рядка з таблиці SQLite, а лише переводить об'єкт у статус `isDeleted: true` . Об'єкти з цим статусом миттєво приховуються з UI, але обов'язково беруть участь у процесах синхронізації логів, щоб видалення синхронізувалося на інші пристрої користувача.
- **Hard Delete (Фізичне очищення):** Повне фізичне видалення записів з локальної бази даних SQLite відбувається автоматично під час фонові процедури обслуговування через 30 днів після виставлення прапора `isDeleted: true` .
- **Захист логів від людського фактора:** Журнал транзакцій (`audit_log.json`) зберігається у прихованій системній папці додатку на Google Drive (`appDataFolder`) у JSON-форматі. Він захищений від прямого доступу користувача через стандартний веб-інтерфейс Google Диска, що унеможливорює випадкове пошкодження структури файлу.

2.2 Деталізація Системи Аудиту (Transaction Log & Event Sourcing)

Для реалізації надійної реплікації даних Filamentary спирається на концепцію **Append-Only Log**. Кожна подія модифікації даних локально записується в окрему структуру.

Приклад згенерованого пакету даних `audit_log.json` , який пристрій відправляє у хмару:

```
{
  "device_id": "desktop_win_11_x64_prod",
  "last_synced_version": 14,
  "events": [
```

```
{
  "event_id": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d",
  "timestamp": 1779849600,
  "entity_type": "MaterialModel",
  "entity_id": "a55c8cfa-42aa-49b8-a032-cdd494d441b9",
  "action": "UPDATE",
  "payload": {
    "usedWeight": 345.5,
    "version": 3,
    "isDeleted": false
  }
},
{
  "event_id": "0c2efc5e-4c8e-5cbe-0cee-3c1e8c4edd7e",
  "timestamp": 1779851200,
  "entity_type": "PrintJob",
  "entity_id": "7850f613-8160-4ed3-b7e3-833842275539",
  "action": "CREATE",
  "payload": {
    "printerId": "8600c0fa-42aa-49b8-a032-cdd494d441b9",
    "modelName": "Bambu_A1_Toolhead_Cover.gcode",
    "startTime": 1779851200,
    "status": "completed"
  }
}
]
```

3. Функціональні вимоги

- **Моніторинг стану:** Безперервне опитування (Polling) стану підключених принтерів за допомогою рекурсивних асинхронних ізольованих запитів через Moonraker API (для Klipper) або REST/MQTT API (для Bambu Lab). Запити виконуються з інтервалом в 3 секунди з автоматичною обробкою таймаутів.
- **Управління матеріалами:** Контроль залишків пластику. Автоматичне списання ваги після успішного завершення друку, а також можливість ручного коригування ваги катушки через UI.

- **Валідація числових полів:** Для виключення збоїв локальної математики та помилок парсингу, на рівні UI та BLoC впроваджується жорстка валідація за регулярним виразом $^{[0-9]*\.\?[0-9]}+\$$. Роздільником десяткових знаків завжди виступає крапка.
- **Авто-завершення катушок:** Катушки, у яких зафіксовано показник $remainingWeight \leq 0$, де $remainingWeight = initialWeight - usedWeight$, автоматично змінюють свій статус з `active` на `finished` і виключаються з переліку доступних матеріалів для заправки слотів.

3.1 Модуль Автоматичного Списання (3D Print & Inventory Sync)

Синхронізація завершених друкарських завдань з інвентарем відбувається за наступним алгоритмом:

1. Під час пулінгу через мережевий клієнт, додаток фіксує зміну статусу принтера з `printing` на `completed` .
2. Додаток робить запит до принтера для завантаження метаданих роздрукованого файлу (наприклад, через ендпоінт `/server/files/metadata?filename=...` у Klipper).
3. З файлу метаданих зчитується параметр `filament_weight_total` (загальна вага пластику в грамах, згенерована слайсером).
4. **Каскадна локальна транзакція:**
 - Додаток перевіряє структуру `active_slots_json` для визначення UUID катушки, яка була активна в процесі друку.
 - Виконується списання ваги: $usedWeight_{new} = usedWeight_{current} + filament_weight_total$.
 - При використанні багатоколірних систем (Bambu AMS) вага розділяється пропорційно на основі логу зміни інструментів (`toolchange log`), якщо пристрій надає такі дані.

4. Синхронізація та вирішення конфліктів

- **Conflict Resolution:** У разі виявлення розбіжностей під час синхронізації (коли один і той самий об'єкт отримав різні зміни на двох пристроях в офлайн-режимі й має однакові версії), додаток відмовляється від автоматичного перезапису та активує режим **User-Driven Conflict Resolution**.
- **Інтерфейс конфліктів:** Користувачеві демонструється модальне вікно або екран з порівняльною таблицею, що містить дві колонки: «Локальна версія на пристрої» та «Хмарна

версія з Google Drive». Користувач має три варіанти дії: «Залишити локальну», «Замінити на хмарну» або «Об'єднати дані» (якщо структура дозволяє мерж).

- **Алгоритм математичної версійності:** Після успішного вирішення конфлікту або при будь-якому штатному оновленні об'єкта, нова версія розраховується за формулою:

$$\text{newVersion} = \max(v_1, v_2) + 1$$

4.1 Трьохкомпонентний Алгоритм Синхронізації (The Merge Engine)

Фонові синхронізація ізольована від основного потоку інтерфейсу користувача (UI Thread) за допомогою Dart Isolate.

1. **Пулл логів (Pull):** Додаток завантажує актуальні файли змін інших пристроїв з ізольованого сховища Google Drive.
2. **Аналіз версій (Scan):** Програма порівнює `version` та `timestamp`.
3. **Вирішення конфліктів за принципом "CRDT-lite":**
 - Якщо `remote.version > local.version` і поля не конфліктують, локальна база оновлюється через `drift.into(tables).insertOrReplace()`.
 - Якщо `remote.version == local.version`, але хмарний `timestamp` є новішим за локальний більше ніж на 500 мілісекунд, система застосовує правило *Last-Write-Wins (LWW)* і перезаписує локальний рядок.

4.2 Специфіка інтеграції Google Drive API

Оскільки авторизація реалізована через комбінацію `Firebase Auth` + `Google Sign-In`, додаток отримує `accessToken` і використовує його для прямих та легких REST-запитів до Google API, уникаючи використання важких сторонніх клієнтських бібліотек.

- **Обмеження області видимості (Scopes):** Додатку заборонено запитувати глобальний доступ до файлів користувача (`drive.file`). Замість цього використовується виключно область видимості `https://www.googleapis.com/auth/drive.appdata`. Це надає додатку доступ лише до його власної прихованої папки на Диску користувача. Користувач не зможе випадково видалити або модифікувати базу через веб-браузер.

5. Нефункціональні вимоги та Error Handling

- **Performance:** Час повної синхронізації журналу транзакцій (до 50 записів) між пристроєм і хмарою не повинен перевищувати 5 секунд у стабільному мережевому середовищі. Час відгуку інтерфейсу користувача на будь-яку дію (кліки, переходи, відкриття вікон) — менше ніж 100 мс.
- **Offline-first:** Додаток зберігає 100% працездатність бізнес-функцій (додавання матеріалів, перегляд історії, редагування принтерів) за повної відсутності інтернет-з'єднання. Всі транзакції накопичуються в локальній черзі чекаючи виходу в мережу.
- **Error Handling та відмовостійкість:**
- **Automated Rollback:** Будь-яка невдала або перервана операція запису в базу даних Drift автоматично викликає відкат транзакції (Rollback) на рівні SQLite для збереження цілісності таблиць.
- **Логуювання помилок:** Використовується системний логер (logger), який записує всі мережеві та внутрішні помилки синхронізації у файл діагностики.
- **Захист від Split-brain:** Запобігання ситуаціям, коли два пристрої створюють паралельні гілки реальності, шляхом постійної покрокової верифікації ланцюжка timestamp та контрольних сум версій.

5.1 Обробка критичних станів (Fail-Safe & Edge Cases)

- **Втрата інтернет-з'єднання під час синхронізації:** Якщо HTTP-запит до Google Drive переривається помилкою мережі, операція маркується як pending і залишається в таблиці sync_queue . Додаток запускає алгоритм експоненціального бекоффу для повторних спроб (через 1 хв, 2 хв, 4 хв, 8 хв, 16 хв) до повного відновлення зв'язку.
- **Офлайн-робота понад 7 днів (Глибокий Split-Brain):** Якщо пристрій підключився до мережі після тривалого офлайну і виявив, що різниця між локальною версією та хмарною становить понад 50 кроків модифікації, автоматичний автоматизований мердж повністю блокується. Система примусово переводить додаток у режим ручного вибору опорної бази даних для уникнення затирання актуальних даних.

6. План реалізації (Roadmap)

Етап	Опис	Термін
1	Проектування реляційної БД та повна інтеграція SQLite (Drift, створення таблиць і зв'язків FK)	1 тиждень
2	Реалізація фонового ядра синхронізації, інтеграція Google Drive API (Scopes, OAuth2, Audit Log)	3 тижні
3	Розробка UI-шару на Flutter за допомогою архітектурного патерну BLOC (Екрани, Картки, Діалоги)	3 тижні
4	Тестування інтеграції, симуляція конфліктів, покриття логіки мержу тестами	2 тижні

7. Безпека та Шифрування

Зберігання конфіденційних даних користувача (IP-адреси серверів, токени доступу, унікальні API-ключі принтерів Vambu Lab) вимагає захисту на фізичному рівні пристрою.

- **Шифрування бази:** Локальний файл SQLite шифрується на льоту за допомогою розширення **SQLCipher**.
- **Збереження ключів:** Ключ дешифрування бази даних генерується випадковим чином під час першого запуску додатку і записується в захищене системне сховище ОС: **Android Keystore** (для смартфонів) та **Windows Credential Manager** (для ПК) за допомогою пакета `flutter_secure_storage`.

8. Структура проекту (Папкова архітектура)

Додаток слідує принципам Feature-First архітектури для забезпечення легкої підтримки коду:

```
lib/  
├── core/ <-- ФУНДАМЕНТ (те, що використовують усі фічі)  
│   ├── database/ <-- Чистий Drift + SQLCipher (Безпека локальних даних)  
│   └── database.dart <-- Ініціалізація та налаштування підключення БД
```


- | | └─ tables.dart <-- Опис реляційних таблиць (Printers, Materials, PrintJobs)
- | └─ di/ <-- Dependency Injection (Налаштування get_it та injectable)
- | └─ network/ <-- Налаштування Dio клієнта, базові інтерцептори для принтерів
- | └─ localization/ <-- Файли перекладів (uk_UA, en_US)
- | └─ theme/ <-- Стилiзація: Світла/Темна теми, палітра кольорів Material 3
- | └─ utils/ <-- Глобальні хелпери та утиліти (AppLogger тощо)
- |
- | └─ features/ <-- ФІЧІ (незалежні бізнес-модулі)
- | |
- | | └─ inventory/ <-- 📦 Фича: Матеріали та Котушки філаменту
- | | | └─ data/ <-- Репозиторій інвентаря, Drift Stream Mappers
- | | | └─ domain/ <-- Сутнісна модель MaterialModel, контракт репозиторію
- | | | └─ presentation/ <-- InventoryScreen, керування станом MaterialBloc
- | |
- | | └─ printers/ <-- 🖨️ Фича: Принтери, Телеметрія та Друк
- | | | └─ data/ <-- Рекурсивний Polling Service, Moonraker/Bambu клієнти
- | | | └─ domain/ <-- Моделі Printer, PrintJob, бізнес-правила
- | | | └─ presentation/ <-- PrintersScreen, Екран кабінету PrinterDetailScreen
- | |
- | | └─ sync/ <-- ☁️ Фича: Децентралізована синхронізація (Local-First ядро)
- | | | └─ data/ <-- Робота з Google Drive REST API, обробка завантажень файлів
- | | | └─ domain/ <-- Обчислювальні алгоритми Merge Engine та версійності
- | | | └─ presentation/ <-- Компонент ConflictGuard, Екран конфліктів для користувача
- | |
- | | └─ onboarding/ <-- 🚀 Фича: Перший запуск та вітання користувача
- | | | └─ data/
- | | | └─ domain/
- | | | └─ presentation/
- | |
- | | └─ settings/ <-- ⚙️ Фича: Налаштування додатка
- | | | └─ data/ <-- GoogleAuthService, SharedPreferences (мова/тема)
- | | | └─ domain/ <-- Контракт SettingsRepository, описи станів AuthState
- | | | └─ presentation/ <-- SettingsScreen, блоки керування ThemeBloc, AuthBloc
- |
- └─ main.dart <-- Точка входу в додаток, ініціалізація та запуск AppGuard